5584d91a-95e2-47e0-af9b-e589ee3a7ff1

<current>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'mission-apprentissage/labonnealternance' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/labonnealternance/labonnealternance (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-05-15T14:08:07.560+00:00</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
fix(lba-3934): corriger l'encodage des entités html dans offer_title et workplace_name
</pr_title>

<pr_description>
https://tableaudebord-apprentissage.atlassian.net/browse/LBA-3934

Problème

Les partenaires envoient parfois des titres d'offres avec des

entités HTML déjà encodées, par exemple :

```
alternance 12 mois – technicien.ne d'études in vitro &amp;amp; in vivo F/H
```

`sanitizeTextField` décodait `&` → `&` via `he.decode`, puis `sanitizeHtml` ré-encodait `&` → `&`. Résultat : la valeur stockée en base contenait `&`, affiché littéralement sur le front partout où `offer_title` est rendu comme texte brut (aria-labels, cellules tableau, etc.).

Changements

- **`sanitizeTextField`** : quand `keepFormat=false`, décode les entités HTML en sortie (`he.decode` après `sanitizeHtml`). La sémantique est maintenant claire : `keepFormat=false` → texte brut, `keepFormat=true` → HTML-safe pour `dangerouslySetInnerHTML`. La sanitization XSS reste intacte (`sanitizeHtml` strip les balises dangereuses avant le decode final).
- **`formatTextFieldsJobsPartners`** : `offer_title` et `workplace_name` passent à `keepFormat=false` – ce sont des champs texte brut, sans HTML formaté.
- **Migration** : décode les entités HTML dans `offer_title` et `workplace_name` directement dans la collection `jobs_partners` pour corriger les données existantes.
- **Tests** : mis à jour pour refléter la nouvelle sémantique de `keepFormat=false`.

Plan de test

- [ ] Vérifier qu'une offre avec `&` dans le titre s'affiche bien avec `&` sur la page de résultats
- [ ] Vérifier que l'aria-label de la carte offre ne contient plus `&`
- [ ] Vérifier qu'une description avec HTML formaté (`<p>`, `<ul>`) s'affiche toujours correctement
- [ ] Lancer la migration et vérifier en base que les titres affectés sont corrigés
- [ ] `yarn test server/src/common/utils/stringUtils.test.ts` → 66 tests passent
</pr_description>

Comments on the PR (you are @copilot in this section)

<comments>

<pr_comments>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
Pull request overview

Ce PR vise à corriger le double-encodage d’entités HTML (ex: `&`) dans des champs texte (`offer_title`, `workplace_name`) afin d’éviter l’affichage littéral des entités côté UI.

Changes:
- Ajuste `sanitizeTextField` pour décoder des entités HTML en sortie lorsque `keepFormat=false`.
- Modifie la sanitation des champs partenaires (`offer_title`, `workplace_name`) pour les traiter comme “texte brut”.
- Ajoute une migration MongoDB pour décoder des entités HTML déjà stockées dans `jobs_partners`.

Reviewed changes

Copilot reviewed 4 out of 4 changed files in this pull request and generated 4 comments.

File	Description
server/src/common/utils/stringUtils.ts	Change la sémantique de `sanitizeTextField` (décodage en sortie si `keepFormat=false`).
server/src/common/utils/stringUtils.test.ts	Met à jour les tests pour refléter la nouvelle sémantique “texte brut”.
server/src/jobs/offrePartenaire/formatTextFieldsJobsPartners.ts	Passe `offer_title` et `workplace_name` en `keepFormat=false`.
server/src/migrations/20260515154140-fix-html-entities-offer_title-workplace_name.ts	Migration de correction des entités HTML en base sur `jobs_partners`.

</comment_old>

</pr_comments>

<pr_reviews>

<comment_thread_resolved>
<file>server/src/common/utils/stringUtils.ts:13-15</file>
<original_commit>d7a6ea0</original_commit>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
Le `he.decode(sanitized)` quand `keepFormat=false` peut réintroduire des balises à partir d'entrées encodées (ex: `<img…>` devient `<img…>`). Or `sanitizeTextField(...)` est utilisé pour alimenter des templates email rendus sans échappement via EJS `<%- ... %>` (ex: `server/static/templates/mail-reponse-cfa.mjml.ejs:89` et `mail-candidat-entretien.mjml.ejs:87`), ce qui permettrait d'injecter du HTML dans les emails. Suggestion: ne pas décoder les entités dans `sanitizeTextField` (garder une sortie HTML-safe), et introduire un helper dédié pour le cas "texte brut pour React" (ou un flag explicite) afin d'éviter une régression XSS sur les usages existants.

</comment_old>

</comment_thread_resolved>

<comment_thread_resolved>
<file>server/src/jobs/offrePartenaire/formatTextFieldsJobsPartners.ts:24-27</file>
<original_commit>d7a6ea0</original_commit>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
`offer_title` et `workplace_name` sont consommés côté UI via `dangerouslySetInnerHTML` (ex: `ui/app/(editorial)/alternance/_components/CarteOffre.tsx:53,66` et `ui/app/(candidat)/(recherche)/recherche/_components/RechercheResultats/LbaItemCard.tsx:38,149`). En les passant à `sanitizeTextField(..., false)`, on obtient désormais une sortie "texte brut" pouvant contenir des `<`/`>` (voire des balises réintroduites via décodage d'entités), ce qui est incompatible avec `dangerouslySetInnerHTML` et ouvre un risque XSS / rendu cassé. À corriger soit en conservant `keepFormat=true` pour ces champs tant que le front les rend en HTML, soit en modifiant le front pour rendre ces champs en texte (sans `dangerouslySetInnerHTML`).
</comment_old>

</comment_thread_resolved>

<comment_thread_resolved>
<file>server/src/migrations/20260515154140-fix-html-entities-offer_title-workplace_name.ts:4-7</file>
<original_commit>d7a6ea0</original_commit>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
La migration ne fait qu’un seul `he.decode()` ; pour une valeur doublement encodée (ex: `&`), on aboutit généralement à `&` après un seul passage, et pas à `&`. Comme l’objectif annoncé est de corriger les titres contenant `&`, il faudrait décoder de façon itérative jusqu’à stabilité (avec un nombre max d’itérations) afin de réellement normaliser les données.

This issue also appears on line 12 of the same file.

</comment_old>

</comment_thread_resolved>

<comment_thread_resolved>
<file>server/src/common/utils/stringUtils.test.ts:49-53</file>
<original_commit>d7a6ea0</original_commit>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
Les tests valident désormais que `sanitizeTextField(..., false)` peut retourner des balises réelles issues d’entités encodées (ex: `<script>` -> `<script>`, et `<img…>` -> `<img…>`), avec l’argument “safe in JSX”. Or la fonction est utilisée côté serveur pour des emails MJML/EJS insérés sans échappement via `<%- ... %>` ; dans ce contexte, ces valeurs deviendraient du HTML actif. Il faudrait soit conserver une sortie HTML-safe pour `sanitizeTextField`, soit introduire un helper distinct pour le rendu React texte brut, et ajuster ces assertions en conséquence (et ajouter un test de régression explicite sur le cas `&`).

This issue also appears on line 109 of the same file.
</comment_old>

</comment_thread_resolved>

</pr_reviews>

<pr_comments>

<comment_new>
<comment_id>4460445389</comment_id>
<author>@kevbarns</author>
@copilot review again
</comment_new>

</pr_comments>

</comments>

The last **2** git commits in this branch are the changes you have made so far. Use those as your change commit history.
</problem_statement><rules_for_making_changes>
* Start by fully understanding <problem_statement> and the <comments> before making any changes. You will only address the <comment_new> comments. Reply to new comments as you address them. Ignore comments that are not actionable or directed at you.
* For each new comment, determine:
    - If it a request for changes, a question on the code or a suggestion.
    - Is it addressed to you or someone else, e.g. "Hey @username, ..." or "@username can you ...".
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you can stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment and ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>

<system_reminder>
You have the capability to call multiple tools in a single response.
For maximum efficiency, whenever you need to perform multiple independent operations, ALWAYS call tools simultaneously whenever the actions can be done in parallel rather than sequentially (e.g. multiple reads/edits to different files). Especially when exploring repository, searching, reading files,

viewing directories, validating changes. For example, you can read 3 different files in parallel, or edit different files in parallel. However, if some tool calls depend on previous calls to inform dependent values like the parameters, do NOT call these tools in parallel and instead call them sequentially (e.g. reading shell output from a previous command should be sequential as it requires the sessionID).
</system_reminder>